Machine Learning for Artificial Intelligence Engineering

Assignment 1

Linear and Logistic Regression, Gradient Descent

Students:
Amer Delić (01331672)

Lecturers:
Umair Khan
Maximilian Baronig

Institute of Machine Learning and Neural Computation
Graz University of Technology

Graz, November 2025

Contents

1 Linear Regression **3**
1.1 Univariate Linear Regression 3
1.2 Multiple Linear Regression 8
1.3 Polynomial Regression 8

2 Logistic Regression **10**

3 Gradient Descent **13**

List of Figures

1.1 Pairplot of smartwatch variables 4
1.2 Duration–calories linear fit 5
1.3 Avg pulse–max pulse linear fit 5
1.4 Duration–fitness level linear fit 5
1.5 Hours sleep–fitness level nonlinear 7
1.6 Hours work–exercise intensity nonlinear 7
1.7 Avg pulse–duration nonlinear 7
1.8 Polynomial regression fit 9

2.1 Decision boundaries of the trained classifier models 12
a Train set predicitions - Dataset 1 12
b Test set predicitions - Dataset 1 12
c Train set predicitions - Dataset 2 12
d Test set predicitions - Dataset 2 12
e Train set predicitions - Dataset 3 12
f Test set predicitions - Dataset 3 12

3.1 Rastrigin function 13
3.2 Rastrigin function - 2D contour plot 14
3.3 Trajectory of the gradient descent steps 15
3.4 Function evolution over iterations 15

1 Linear Regression

1.1 Univariate Linear Regression

To get an initial overview of the dataset, a pairplot of all eight smartwatch variables was generated. It shows the distributions of each feature and the scatter plots between all feature pairs. This plot was used to visually identify which variable pairs may show a linear relationship.

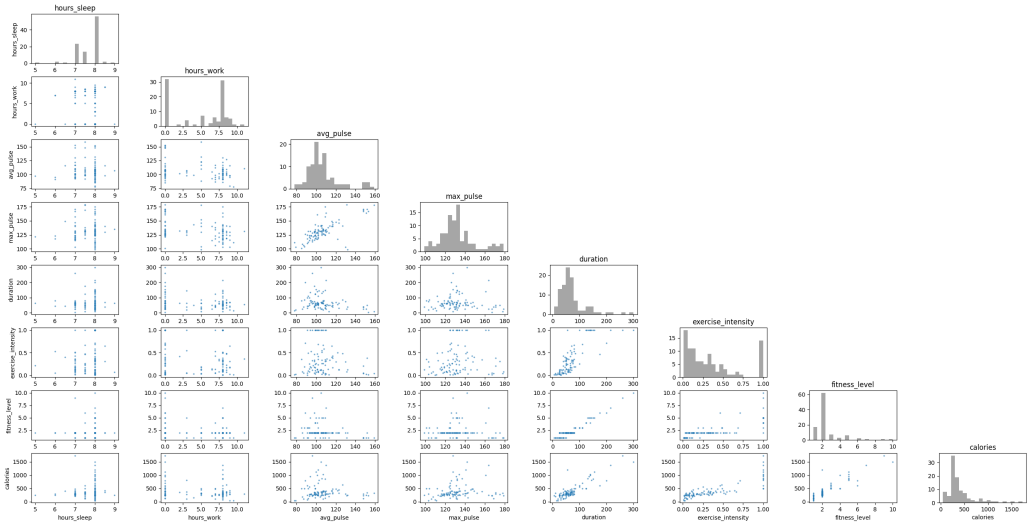


Figure 1.1: Pairplot of all variables in the smartwatch dataset.

In this part of the assignment, the goal is to check which pairs of variables in the smartwatch dataset show a linear relationship. For each selected pair, a univariate linear regression model is fitted, the mean–squared error (MSE) and the Pearson correlation coefficient are computed, and the data is plotted together with the fitted line.

Based on the pairplot, three pairs of variables were selected that appear to have a clear linear relationship: *duration–calories*, *avg_pulse–max_pulse*, and *duration–fitness_level*.

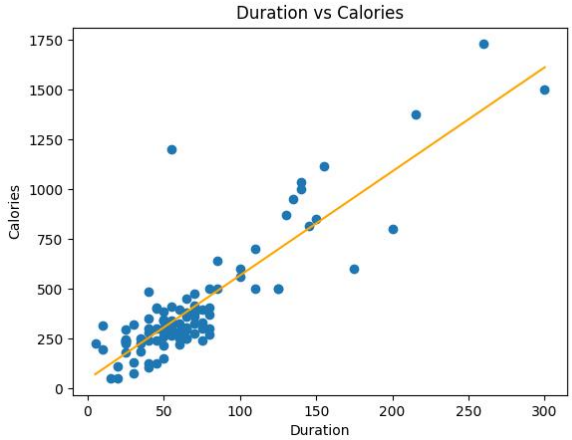


Figure 1.2: Scatter plot of *duration* and *calories* with fitted linear model.

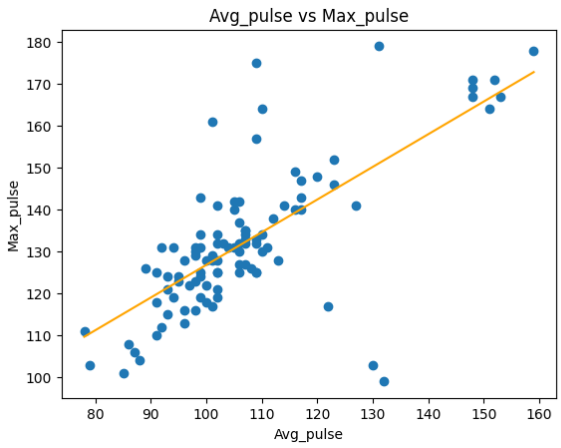


Figure 1.3: Scatter plot of *avg_pulse* and *max_pulse* with fitted linear model.

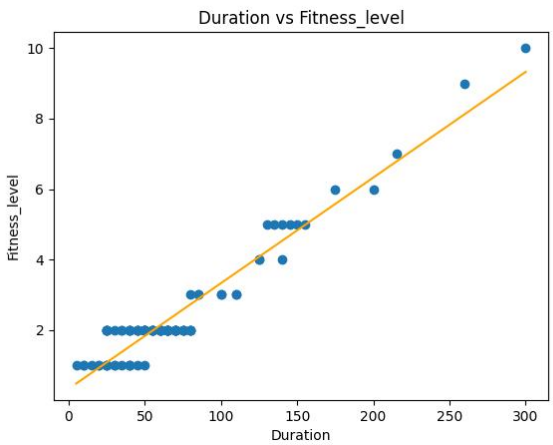


Figure 1.4: Scatter plot of *duration* and *fitness_level* with fitted linear model.

For the three selected pairs that showed a linear relationship, the fitted models produced low errors and high Pearson correlation values. For the *duration–calories* pair, a strong positive correlation was observed ($p = 0.87$), with parameters $\theta^* = [46.24,\ 5.22]$ and an MSE of 21843. The *avg_pulse–max_pulse* pair also showed a clear linear trend, with $p = 0.72$, $\theta^* = [48.87,\ 0.78]$, and an MSE of 144.59. The strongest linear relationship was found for the *duration–fitness_level* pair ($p = 0.96$), where the model parameters were $\theta^* = [0.33,\ 0.03]$ and the error was very small (MSE = 0.20).

In addition to these, three variable pairs that do not show a meaningful linear relationship were also selected: *hours_sleep–fitness_level*, *hours_work–exercise_intensity*, and *avg_pulse–duration*.

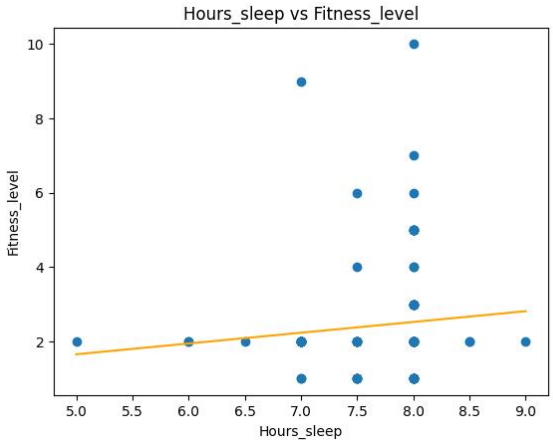


Figure 1.5: Scatter plot of *hours_sleep* and *fitness_level* with fitted linear model.

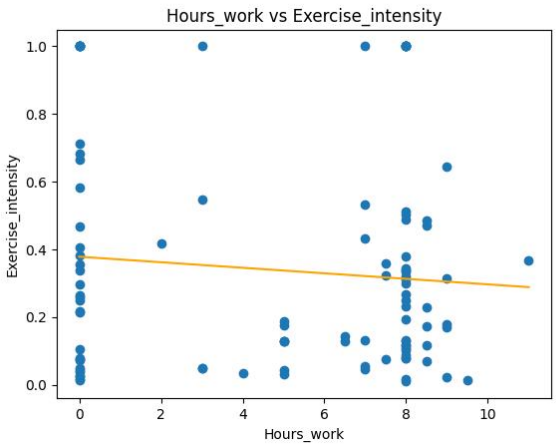


Figure 1.6: Scatter plot of *hours_work* and *exercise_intensity* with fitted linear model.

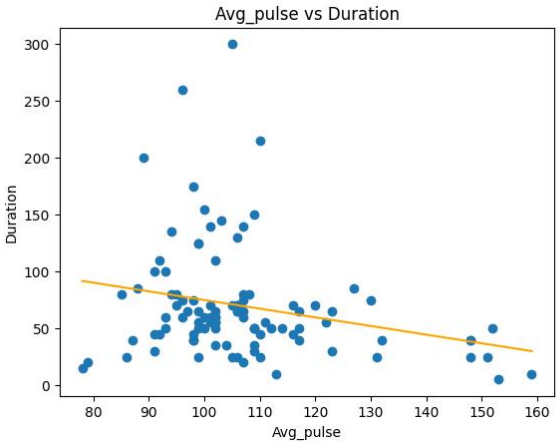


Figure 1.7: Scatter plot of *avg_pulse* and *duration* with fitted linear model.

In contrast, the three pairs selected as non-linear showed very low or weak correlations, and the fitted linear models resulted in higher or less meaningful errors. For the *hours_sleep–fitness_level* pair, the correlation was very small ($p = 0.11$) with $\theta^* = [0.21,\ 0.29]$ and an MSE of 2.40. The *hours_work–exercise_intensity* pair showed almost no linear dependence ($p = -0.10$), resulting in parameters $\theta^* = [0.38,\ -0.01]$ and an MSE of 0.099. Finally, the *avg_pulse–duration* pair had a weak negative correlation ($p = -0.24$), with $\theta^* = [151.19,\ -0.76]$ and a relatively high error (MSE = 2324), confirming the absence of a meaningful linear trend.

1.2 Multiple Linear Regression

For multiple linear regression, three features were used: *duration*, *exercise_intensity*, and *avg_pulse*, with *calories* as the target. The model produced the parameter vector

$$\theta_M = [-194.92,\ 3.61,\ 357.28,\ 2.18]$$

and an MSE of

$$\mathrm{MSE}_M = 15394.69.$$

For comparison, a univariate model using only *duration* to predict *calories* was fitted. This model resulted in

$$\theta_U = [46.24,\ 5.22], \qquad \mathrm{MSE}_U = 21843.03.$$

The multiple regression achieved a lower error than the univariate model, indicating a better fit to the data.

1.3 Polynomial Regression

In the final part of Task 1, a polynomial regression model was fitted in order to capture potential nonlinear patterns that a simple linear model cannot represent. The feature *duration* was used as the input, and *calories* was chosen as the target variable. A polynomial design matrix of degree $K = 4$ was constructed, and the parameters were estimated using the closed-form solution based on the pseudoinverse.

The fitted polynomial model produced the parameter vector

$$\theta_{\text{poly}} = \begin{bmatrix} 166.15 \\ 1.3005 \\ 0.02779 \\ -2.59 \times 10^{-5} \\ -9.59 \times 10^{-8} \end{bmatrix}$$

and the mean–squared error was

$$\mathrm{MSE}_{\text{poly}} = 20717.56.$$

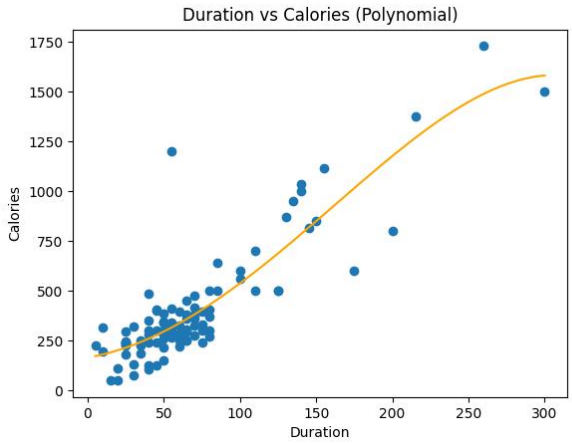


Figure 1.8: Scatter plot of *duration* and *calories* with a fitted polynomial regression model of degree $K = 4$.

Compared to the linear model, the polynomial regression did not provide a notably lower error, indicating that a fourth-degree polynomial does not improve the fit for this feature–target pair in a meaningful way.

2 Logistic Regression

The first step in the logistic regression task was to construct an appropriate design matrix $\mathbf{X}$. For the `Data set` 1 and `Data set 2` the same design matrix was used and it was constructed to include the original features along with quadratic and interaction features, as shown below

$$X_1 = X_2 = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & (x_1^{(1)})^2 & (x_2^{(1)})^2 & x_1^{(1)}x_2^{(1)} \\ x_1^{(2)} & x_2^{(2)} & (x_1^{(2)})^2 & (x_2^{(2)})^2 & x_1^{(2)}x_2^{(2)} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ x_1^{(N)} & x_2^{(N)} & (x_1^{(N)})^2 & (x_2^{(N)})^2 & x_1^{(N)}x_2^{(N)} \end{bmatrix}$$

Design matrix for `Data set 3` was constructed by expanding the original features by a polynomial basic function up to order 4, as shown below

$$X_3 = \begin{bmatrix} X_1 & X_3' \end{bmatrix}$$

where

$$X_{3'} = \begin{bmatrix} (x_1^{(1)})^3 & (x_1^{(1)})^2x_2^{(1)} & x_1^{(1)}(x_2^{(1)})^2 & (x_2^{(1)})^3 & (x_1^{(1)})^4 & (x_1^{(1)})^3x_2^{(1)} & (x_1^{(1)})^2(x_2^{(1)})^2 & x_1^{(1)}(x_2^{(1)})^3 & (x_2^{(1)})^4 \\ (x_1^{(2)})^3 & (x_1^{(2)})^2x_2^{(2)} & x_1^{(2)}(x_2^{(2)})^2 & (x_2^{(2)})^3 & (x_1^{(2)})^4 & (x_1^{(2)})^3x_2^{(2)} & (x_1^{(2)})^2(x_2^{(2)})^2 & x_1^{(2)}(x_2^{(2)})^3 & (x_2^{(2)})^4 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ (x_1^{(N)})^3 & (x_1^{(N)})^2x_2^{(N)} & x_1^{(N)}(x_2^{(N)})^2 & (x_2^{(N)})^3 & (x_1^{(N)})^4 & (x_1^{(N)})^3x_2^{(N)} & (x_1^{(N)})^2(x_2^{(N)})^2 & x_1^{(N)}(x_2^{(N)})^3 & (x_2^{(N)})^4 \end{bmatrix}$$

After splitting each dataset into training and test subsets, a `LogisticRegression` classifier model was created and fitted using the training data. This led to the parameter vectors (bias term included at index 0)

$$\theta_1 = [-38.027,\ 4.092,\ 1.941,\ -0.023,\ -0.027,\ -0.152]$$

$$\theta_2 = [388.842,\ 0.002,\ -0.028,\ -0.656,\ -0.654\ 0.0221]$$

$$\theta_3 = \begin{bmatrix} -3.598 & -0.296 & -2.501 & -2.709 & 1.878 & 2.497 & -0.249 & 0.028 \\ -1.735 & -1.735 & 0.775 & -1.687 & -1.263 & 1.023 & 0.667 & \end{bmatrix}$$

The performance of the model was then evaluated by calculating accuracy and cross-entropy loss on both the training and test sets. The results are summarized in Table 2.1 and Table 2.2. For all experiments, the ***lbfgs*** solver was used. In addition, ***L2*** regularization was applied,

particularly for `Data set 3`, to address the higher complexity of the expanded polynomial feature set and reduce the risk of overfitting.

Dataset	**Train Accuracy (%)**	**Test Accuracy (%)**
Dataset 1	99.53	98.75
Dataset 2	100.00	100.00
Dataset 3	92.08	95.80

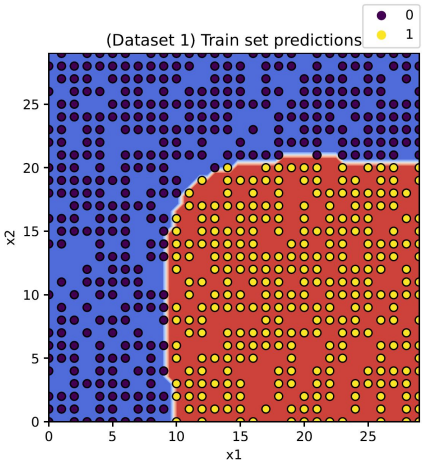
Table 2.1: Comparison of training and test accuracies for the three datasets

Dataset	**Train Loss**	**Test Loss**
Dataset 1	0.03603	0.05961
Dataset 2	0.00043	0.00090
Dataset 3	0.18460	0.14240

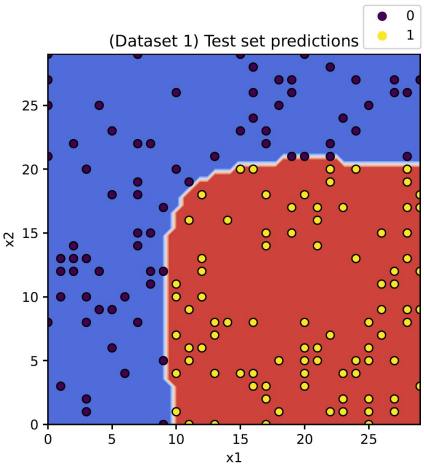
Table 2.2: Comparison of training and test cross-entropy losses for the three datasets

While both accuracy and cross-entropy loss are metrics to evaluate a classifier model - as can be seen above - 100% accuracy does not imply that cross-entropy loss is 0 (e.g. model of the data set 2). Accuracy only checks if the predicted class is correct, while cross-entropy also measures "confidence" of the prediction. Since cross-entropy is defined as the sum of log-probabilities assigned to the true classes, the loss will only be exactly 0 if the model assigns probability 1 to the correct class for every sample — a scenario that is extremely rare in practice.

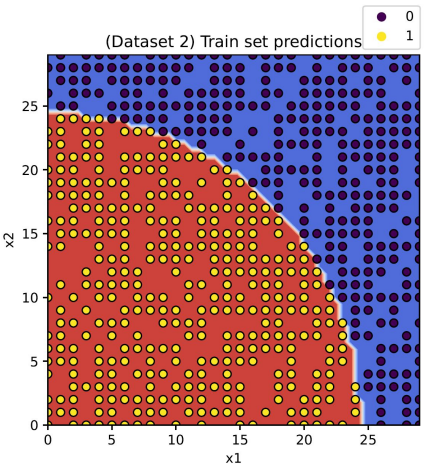
Finally, to visualize the performance of the trained classifier models, the decision boundaries were plotted for each model on both the training and test sets, together with the data points from the respective sets. The resulting visualizations are shown in Figure 2.1.



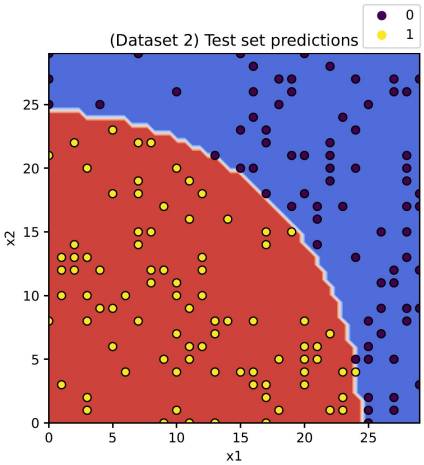
a) Train set predicitions - Dataset 1



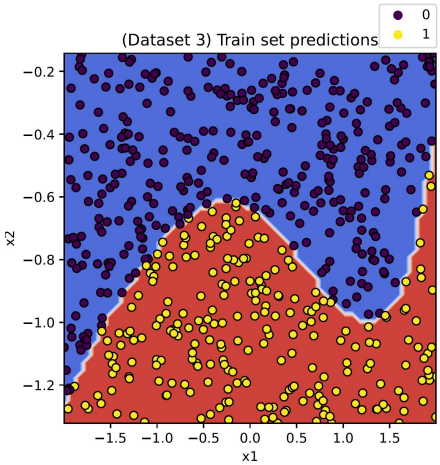
b) Test set predicitions - Dataset 1



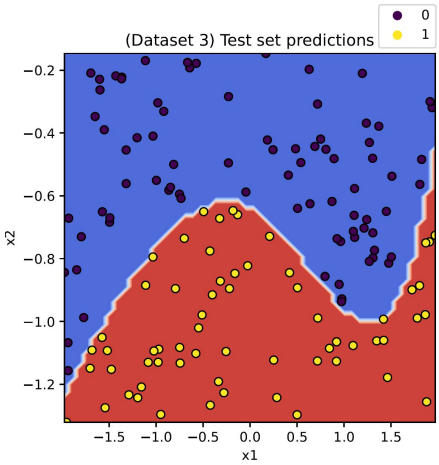
c) Train set predicitions - Dataset 2



d) Test set predicitions - Dataset 2



e) Train set predicitions - Dataset 3



f) Test set predicitions - Dataset 3

Figure 2.1: Decision boundaries of the trained classifier models

3 Gradient Descent

The Rastrigin function shown in Figure 3.1 and Figure 3.2 is optimized using gradient descent in this task. As can be observed, the function has a highly non-convex and oscillating surface, which does not guarantee convergence to the global minimum and often results in misleading behavior and rapidly changing gradient directions. In addition, the function contains a large number of regularly spaced local minima, so the algorithm often becomes trapped in a local minimum instead of reaching the global minimum. Consequently, the starting point strongly influences the outcome, as different initializations lead to convergence toward different minima. Taken together, these properties make the Rastrigin function extremely challenging to optimize using gradient descent.

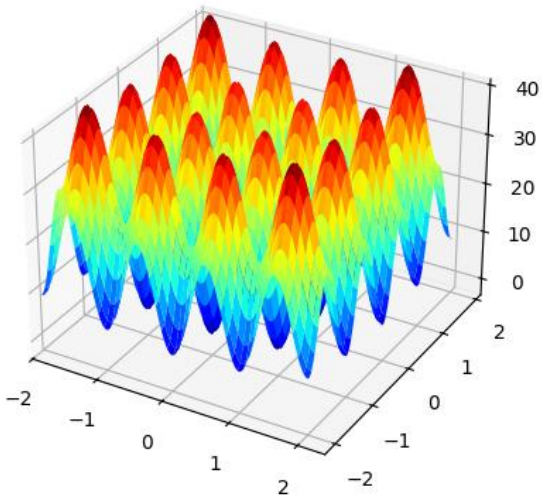


Figure 3.1: Rastrigin function

After implementing the gradient descent algorithm, different sets of hyperparameters — learning rate, learning decay rate, and number of iterations — were tested. While most configurations resulted in unsatisfactory convergence and (poor) local minima, it was shown that the global minimum can be reached using initial learning rate

$$\eta_0 = 0.02$$

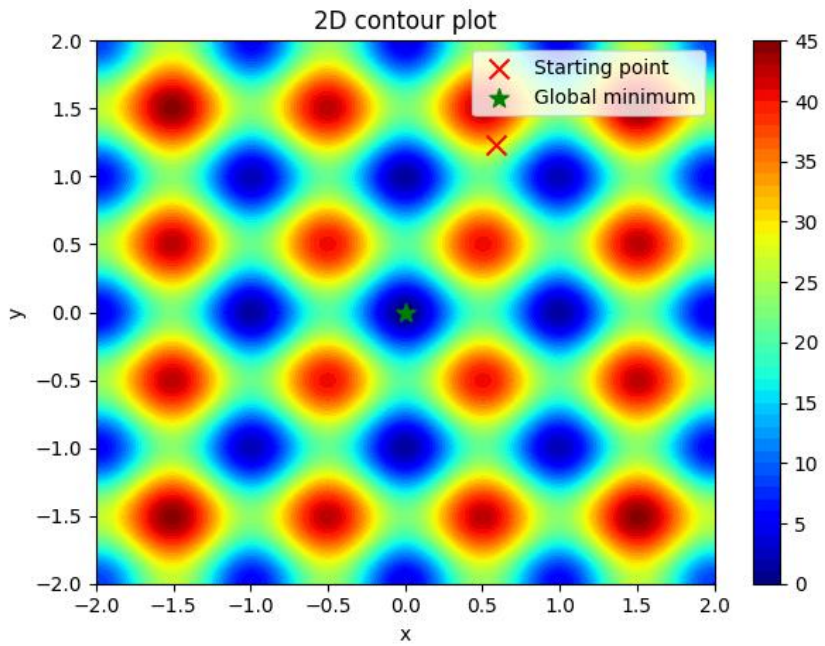


Figure 3.2: Rastrigin function - 2D contour plot

and learning decay rate

$$r = 0.6$$

As can be observed in Figure 3.3 and Figure 3.4, using the specified hyperparameter set for the chosen starting point, the global minimum is reached surprisingly quickly.

Furthermore, the importance of using a learning rate decay can be observed. Due to the highly oscillating surface of the Rastrigin function, a fixed learning rate would often cause the gradient descent algorithm to become trapped in (poor) local minima, as the learning rate remains too large to refine the solution. It may also lead to indefinite oscillations around minima instead of converging, or even to divergence in steep regions. With a fixed low learning rate, convergence may eventually occur, but progress will be extremely slow and the algorithm may still become trapped in local minima.

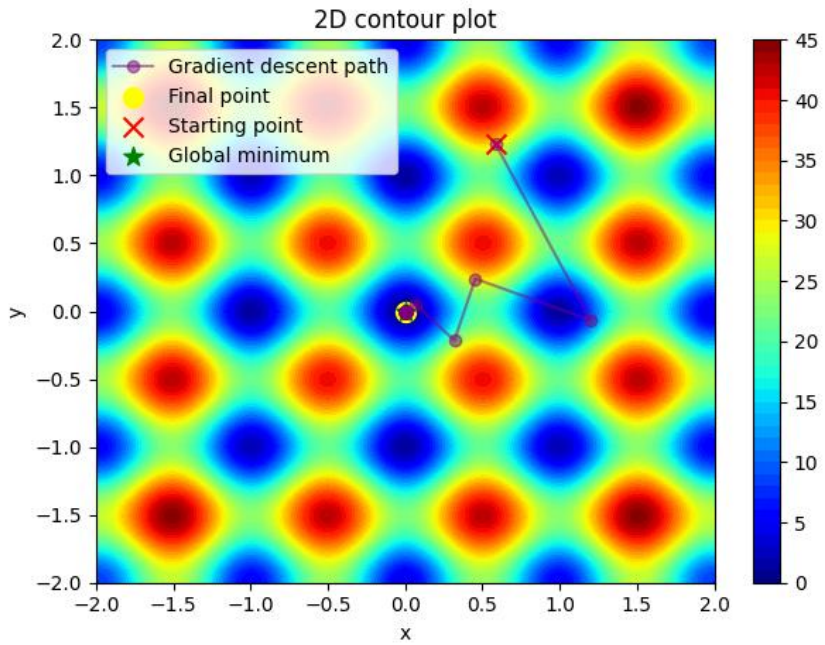


Figure 3.3: Trajectory of the gradient descent steps

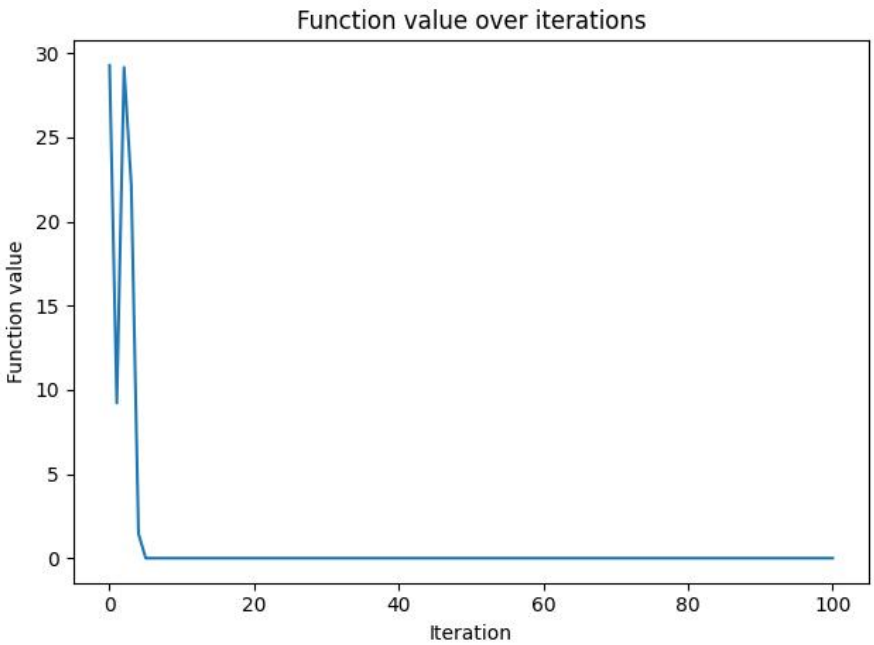


Figure 3.4: Function evolution over iterations